

Week 1 Project —

Prompt Manager System

Two-service architecture: Prompt Store + Prompt Review | June 2025

Track	AI Engineering Internship — ATS Internal
Week	Week 1 — Foundations
Services	prompt-service + review-service
Stack	FastAPI · SQLite · httpx · Pydantic v2 · python-dotenv
Prepared By	Usaid Ahmed Adept Tech Solutions

OVERVIEW

The Week 1 project is a two-service Prompt Manager system. Rather than a toy example, it is a minimal but real multi-service architecture — the same pattern used across every ATS microservice. By the end of the week the intern will have built and wired together two independently running FastAPI services that communicate over HTTP.

The two services are:

- prompt-service — stores, retrieves, updates, and deletes prompts in a SQLite database.
- review-service — fetches a prompt by ID from prompt-service and allows a reviewer to attach a score and written feedback.

Every concept from the Week 1 syllabus — HTTP methods, path/query parameters, Pydantic validation, file/db persistence, outbound httpx calls, UUIDs, and environment variables — appears naturally inside this project. Nothing is forced.

WHY THIS PROJECT COVERS EVERYTHING

The table below maps each Week 1 learning objective to the exact part of the project that exercises it.

Learning Objective	Where It Appears
HTTP methods (GET, POST, PUT, DELETE)	All four used across prompt-service endpoints
Path parameters	GET /prompts/{id}, DELETE /prompts/{id}, GET /reviews/{prompt_id}
Query parameters	GET /prompts?tag=marketing&limit=10
Pydantic request body validation	PromptCreate, PromptUpdate, ReviewCreate models
Persistence — SQLite database	prompt-service stores all prompts in prompts.db

UUIDs for record identification	Every prompt gets a UUID primary key on creation
Environment variables + .env	DB path, service port, review-service base URL
Outbound HTTP with httpx	review-service calls GET /prompts/{id} on prompt-service
Error handling — HTTPException	404 when prompt not found, 400 on bad input, 503 if prompt-service is down
Service-to-service communication	review-service is a consumer of prompt-service's HTTP API

SERVICE 1 — PROMPT-SERVICE

Runs on port 8000. Owns all prompt data. No other service writes to its database — only prompt-service does. Other services must go through its API.

Data Model (Tentative change it according to will)

A Prompt record contains the following fields:

- id — UUID, generated on creation, primary key
- name — short human-readable label for the prompt (e.g. "Cold Email Opener v2")
- description — optional one-line summary of what the prompt is for
- content — the full prompt text, no length limit
- tags — comma-separated string of tags for filtering (e.g. "sales,outbound")
- model_target — optional field: which LLM this prompt is designed for
- created_at — ISO timestamp, set on creation
- updated_at — ISO timestamp, updated on every PUT

API Endpoints

Method	Endpoint	Description
POST	/prompts	Create a new prompt. Returns the full prompt object with generated UUID.
GET	/prompts	List all prompts. Supports ?tag= and ?limit= query params for filtering.
GET	/prompts/{id}	Fetch a single prompt by its UUID. Returns 404 if not found.
PUT	/prompts/{id}	Update name, description, content, tags, or model_target. Partial update supported.
DELETE	/prompts/{id}	Hard delete a prompt by UUID. Returns 404 if not found.
GET	/prompts/{id}/exists	Lightweight existence check — returns {exists: true/false}. Used by review-service.

Storage

Uses SQLite via Python's built-in sqlite3 module — no ORM, no external dependency. The database file path is read from the DB_PATH environment variable in .env. The intern must write raw SQL: CREATE TABLE, INSERT, SELECT, UPDATE, DELETE. This is intentional — it builds foundational understanding before any ORM abstraction is introduced.

Technical Requirements

- Service starts with: `uvicorn main:app --reload --port 8000`
- DB_PATH, SERVICE_PORT loaded from .env via python-dotenv
- Database and table are created on startup if they do not exist
- All endpoints tested via Swagger UI at /docs before marking as complete
- No hardcoded strings — all config via environment variables

SERVICE 2 — REVIEW-SERVICE

Runs on port 8001. Does not have its own prompt database. To fetch prompt content, it calls prompt-service over HTTP using httpx. This is the core architectural lesson: review-service is a consumer of prompt-service's API, not a peer that shares a database.

Data Model (Tentative change it according to will)

A Review record contains the following fields:

- id — UUID, generated on creation
- prompt_id — UUID of the prompt being reviewed (foreign reference — not a DB foreign key)
- prompt_snapshot — the full prompt content at time of review, copied from prompt-service response
- reviewer_name — name of the person submitting the review
- score — integer 1–5
- feedback — written notes from the reviewer
- reviewed_at — ISO timestamp

How It Works

When a review is submitted, review-service does the following in sequence:

- Receives POST /reviews with {prompt_id, reviewer_name, score, feedback} in the request body
- Calls GET /prompts/{prompt_id} on prompt-service using httpx to fetch the prompt
- If prompt-service returns 404 — returns 404 to the caller with a clear error message
- If prompt-service is unreachable — returns 503 Service Unavailable
- Copies the prompt content into prompt_snapshot on the review record
- Saves the review to its own local JSON file store
- Returns the full review object including the snapshot

API Endpoints

Method	Endpoint	Description
POST	/reviews	Submit a review for a prompt by ID. Fetches prompt from prompt-service.
GET	/reviews	List all reviews. Supports ?prompt_id= to filter by prompt.

GET	/reviews/{id}	Fetch a single review by its UUID.
GET	/reviews/{prompt_id}/summary	Return average score and all feedback for a given prompt_id.

Storage

Review-service stores reviews as individual JSON files in a /reviews directory — one file per review, named by UUID. This intentionally contrasts with the SQLite approach in prompt-service, so the intern experiences both persistence patterns in the same week.

Technical Requirements

- Service starts with: `uvicorn main:app --reload --port 8001`
- `PROMPT_SERVICE_URL` loaded from `.env` (e.g. `http://localhost:8000`)
- All `httpx` calls wrapped in `try/except` — network errors must not crash the service
- Score must be validated as integer between 1 and 5 via Pydantic Field constraints
- `prompt_snapshot` stored verbatim — if the original prompt is later edited, the review still reflects what was reviewed

DELIVERABLES

Deliverable	Description
Architecture Diagram	Hand-drawn or typed. Both services, ports, HTTP call, storage.
prompt-service	Fully functional FastAPI + SQLite service with all 6 endpoints.
review-service	FastAPI service with <code>httpx</code> integration, JSON storage, 4 endpoints.
Integration Test	Both services running, end-to-end flow demonstrated live.
README	Setup guide, <code>.env</code> config, and example <code>curl</code> /Swagger requests.
Presentation	Live demo + explanation of every design decision to mentor.

RULES OF ENGAGEMENT

- Both services must run simultaneously for the final demo — not one at a time.
- No database sharing between services. `review-service` may not import or touch `prompt-service`'s DB file.
- No API keys hardcoded. All configuration via `.env` files.
- Git commits required after each day. No commit = work did not happen.
- AI tools are allowed. The intern must be able to explain every line in the final review.

- If stuck for more than 30 minutes, ask for help. Struggling silently is not a virtue here.

Prepared by Adept Tech Solutions — AI Engineering Division | June 2025
This document is for internal use only and is subject to revision by the program lead.